

GRAPHICAL LANGUAGES

- 8.1 Need for Machine Independent Graphical Languages
- 8.2 Graphical Languages: PHIGS, GKS
- 8.3 Graphics Software Standard
- 8.4 Overview of Graphics File Formats
- 8.5 Data Structure in Computer Graphics
- 8.6 Introduction to Open GL

Graphics file format

A graphics file format refers to the structure and encoding of a digital image file. It determines how the image data is stored, compressed, and displayed. There are numerous graphics file formats available, each with its own characteristics, advantages, and specific uses.

Reading Graphics File Data

Image data are read from a Graphics File.

Compressed image data is read one byte at a time from file and into memory before it is decompressed.

Uncompressed image data is often stored only as bytes, even when the pixels are two three or four bytes in size.

Writing a Graphics File Data

Inverse of reading a graphics file

Writers may send data directly to an output device, such as a printer or they may create image files and store data in them.

- **Lossless compression:** no data is lost in compression, but file sizes are not greatly reduced.
- **Lossy compression:** file size is reduced significantly, but some data is lost in the process.

Types

JPEG(Joint Photographic Experts Group)

- Lossy compression format suitable for realistic images and photographs.
- Provides high levels of compression, resulting in smaller file sizes.
- Quality degradation occurs with each subsequent save due to lossy compression.

- Widely supported and compatible with a variety of software and devices.
- Not suitable for images with sharp lines, text, or graphics due to artifacts introduced by compression.
- Supports millions of colors and variable levels of compression.
- Commonly used for web images and digital photography.
- Balances file size and image quality by allowing users to adjust compression levels.

PNG (Portable Network Graphics)

- Lossless compression format ideal for images with sharp lines, text, or graphics.
- Supports transparency, allowing for the preservation of background details.
- Larger file sizes compared to JPEG due to lossless compression.
- Suitable for web graphics, logos, and images that require high quality and transparency.
- Widely supported and commonly used in both online and offline environments.
- Offers options for different compression levels, including lossless and lossy variants (e.g., PNG-8 and PNG-24).
- Supports various color depths, including grayscale, indexed color, and true color.
- Uses lossless compression algorithms, preserving the original image quality.

GIF (Graphics Interchange Format)

- Supports animation by storing multiple frames within a single file.
- Limited color palette of up to 256 colors, making it suitable for simple graphics and animations.
- Supports transparency, allowing for background removal.
- Relatively small file sizes, making it ideal for web graphics and animations.
- Not suitable for high-quality images or photographs due to the limited color palette.
- Supports simple interlacing for progressive rendering.
- Popular for creating small animated images, such as banners or avatars.
- Supports simple and efficient lossless compression.

TIFF (Tagged Image File)

- Lossless compression format that preserves the original image quality.
- Supports high bit-depth images and can store multiple images or layers.
- Commonly used in professional environments for archiving and printing purposes.
- Large file sizes compared to compressed formats like JPEG or PNG.
- Compatible with various software applications and supports advanced features like transparency and layers.
- Offers options for different compression schemes, including LZW, ZIP, and lossless JPEG.
- Supports various color spaces, including RGB, CMYK, and grayscale.
- Suitable for high-quality images, such as scanned documents or print-ready graphics.

BMP (Bitmap)

- Simple and uncompressed file format used mainly in Windows environments.
- Supports various color depths, including monochrome, 8-bit, and 24-bit true color.
- Widely supported by different software applications.
- Does not employ compression, resulting in large file sizes.
- Suitable for storing images without any loss in quality.
- Provides easy and direct access to pixel data.
- Commonly used for basic images and icons.
- Does not support transparency or advanced features like layers.

SVG:

- Vector-based file format that uses mathematical equations to describe graphics.
- Resolution-independent, allowing for scaling without loss of quality.
- Supports transparency, layers, shapes, and text.
- Suitable for web-based graphics, icons, and illustrations.
- Can be edited and manipulated using vector graphics software.
- Supports interactivity and animation through scripting and CSS styling.

- XML-based format, making it easily readable and editable by humans.
- Provides scalability and flexibility across different devices and screen resolutions.

RAW

- RAW is an uncompressed and unprocessed image file format that contains raw sensor data from a digital camera.
- It preserves maximum image information, providing flexibility for post-processing.
- RAW files do not apply in-camera processing, allowing photographers full control over the final image output.
- RAW files are specific to each camera model and manufacturer.
- RAW files have larger file sizes due to including all raw sensor data.
- They store 12-bit or 14-bit data per color channel, offering a broader tonal range.
- Professional photographers prefer shooting in RAW for maximum quality and creative control.
- RAW files require specialized software to view, process, and convert into compatible formats.

TGA (Truevision Graphics Adapter)

- TGA is a raster graphics file format originally developed by Truevision Inc.
- It supports both uncompressed and compressed image data.
- TGA files can store a variety of pixel depths, including 8-bit, 16-bit, 24-bit, and 32-bit.
- The format supports multiple color spaces, including grayscale, indexed color, and true color.
- TGA files can include alpha channel information, allowing for transparency.
- TGA supports run-length encoding (RLE) compression for efficient storage of image data.
- It is a commonly used format for video game graphics, animations, and texture maps.
- TGA files can store additional data such as image attributes, text, and developer-specific information.

PSD (Photoshop Document)

- Native file format of Adobe Photoshop.

- Supports layers, transparency, and specific Photoshop image adjustments.
- Allows non-destructive editing and maximum flexibility within Photoshop.
- Stores image data, layer styles, masks, and other Photoshop-specific elements.
- Results in large file sizes due to multiple editable layers and image data.
- Compatible with Adobe's suite of software applications.

Graphics data structure

In computer graphics, a graphical data structure refers to a data structure specifically designed and optimized for representing and manipulating graphical elements, such as geometric objects, images, or scenes. These data structures are used to efficiently store and process graphical data for rendering and display purposes.

Graphical data structures are used in computer graphics to efficiently represent, store, and manipulate graphical elements. They enable fast rendering, efficient data representation, visualization, interaction, spatial partitioning, collision detection, scalability, and flexibility in graphics applications.

- Graphical data structures optimize the representation and storage of graphical elements.
- They facilitate efficient rendering and real-time performance in graphics applications.
- Graphical data structures support visualization and interaction with graphical elements.
- Spatial partitioning techniques, such as BVHs and octrees, enable efficient organization and culling of objects.
- Collision detection benefits from graphical data structures for accurate and fast intersection tests.
- Graphical data structures provide scalability and flexibility for handling complex scenes.
- They enhance the efficiency, visual quality, and user experience in computer graphics applications.

Main objectives of GDS

1. **Efficient Data Representation:** Graphical data structures aim to provide efficient storage and representation of graphical elements, optimizing memory usage and facilitating fast access and manipulation of the data.

2. **Rendering Performance:** Graphical data structures are designed to improve rendering performance by enabling efficient traversal and processing of graphical data. They help reduce redundant computations and enable faster rendering of complex scenes.
3. **Visualization and Interaction:** The objective of graphical data structures is to provide a framework for visualizing and interacting with graphical elements. They allow for efficient organization, management, and transformation of objects, facilitating interactive and dynamic experiences.
4. **Spatial Partitioning and Culling:** Graphical data structures assist in spatial partitioning techniques, such as bounding volume hierarchies (BVH) or octrees, for efficient organization and culling of objects. This helps optimize rendering performance by reducing the number of objects that need to be processed and rendered.
5. **Collision Detection:** Graphical data structures aim to facilitate efficient collision detection algorithms by organizing graphical elements spatially. This allows for accurate and fast intersection tests between objects, enabling realistic interactions and simulations.
6. **Scalability and Flexibility:** The objective of graphical data structures is to provide scalability and flexibility, allowing for the handling of large and dynamic scenes. They can accommodate various levels of complexity, support different rendering techniques, and allow for easy modification and manipulation of graphical data.
7. **Optimization and Resource Management:** Graphical data structures aim to optimize resource usage, including memory, computational power, and rendering resources. They enable efficient utilization of available resources, leading to improved performance and responsiveness.

Graphical Language

Graphical language refers to a system of communication that utilizes visual elements, symbols, and diagrams to convey information, ideas, or concepts. It is a form of non-verbal communication that relies on visual representations rather than written or spoken words. Graphical language is commonly used in various fields, including design, engineering, architecture, mathematics, computer science, and data visualization.

Key characteristics of graphical language include:

- **Visual Representation:** Graphical language relies on visual elements such as icons, symbols, shapes, lines, colors, and images to convey meaning. These visual representations are carefully designed to communicate specific information or concepts.

- **Abstraction and Simplification:** Graphical language often simplifies complex ideas or information by abstracting them into visual forms. It allows for the distillation of complex concepts into easily understandable visuals, aiding in comprehension and communication.
- **Universal Communication:** Graphical language has the advantage of being universally understood across cultures and languages. Visual representations can transcend linguistic barriers and convey meaning to a broader audience.
- **Expressive and Engaging:** Graphical language has the ability to evoke emotions, engage the viewer, and enhance the communication experience. It can convey information in a visually appealing and memorable manner.

Graphical language plays a crucial role in effectively communicating complex ideas, information, and instructions in a concise and visually appealing manner. It enables efficient comprehension, engagement, and interpretation of visual communication across various domains and disciplines.

PHIGS (Programmer's Hierarchical Interactive Graphics System):

- PHIGS is a graphics standard developed in the 1980s for interactive computer graphics programming.
- It provides a hierarchical graphics model, allowing objects to be organized in a tree-like structure.
- PHIGS supports 2D and 3D graphics with advanced features like hidden surface removal and shading.
- It focuses on non-photorealistic rendering techniques, allowing for artistic and expressive visual effects.
- PHIGS offers a range of graphical attributes such as colors, line styles, fonts, and transformations.
- It provides extensive control over the appearance and behavior of graphical elements within the hierarchical structure.
- PHIGS supports interactive input handling and event management, enabling user interaction in graphics applications.
- It has functionality for handling complex scenes, including segment management and traversal operations.
- PHIGS is commonly used in fields like CAD (Computer-Aided Design) and scientific visualization.
- Although PHIGS had a significant impact on computer graphics development, it has been largely superseded by more modern standards like OpenGL.

General structure of a PHIGS program

```
OpenPhigs (errorFile, memorySize)
OpenWorkstation (ws, Connection, type) {
    Create and display picture
}
CloseWorkstation (ws)
ClosePhigs
```

Parameter errorfile is to contain any error messages that are generated
memorysize specifies the size of the internal storage area.

Workstation identifier (as integer) as ws connection states the access
mechanism for the workstation.

Type specifies the particular category such as input, output device a
combination "out in" device or an input output metafile.

GKS (Graphical Kernel System):

- GKS is an international standard for computer graphics programming developed in the 1980s.
- It provides a set of basic graphics primitives for 2D and 3D rendering.
- GKS aims to be hardware and software independent, allowing for portability across different computer systems.
- It offers a device-independent graphics model and standard graphical attributes.
- GKS supports transformations, clipping, and basic rendering techniques like solid fill and shading.
- It includes input handling functions for keyboard and pointing device events.
- GKS is designed to support various output devices such as displays, plotters, and printers.
- It provides segmentation capabilities, allowing grouping of graphics primitives for efficient redrawing and management.
- GKS played a significant role in establishing standardized methods for computer graphics programming.
- Although GKS has been largely replaced by more modern graphics technologies, it paved the way for subsequent graphics standards and frameworks.

GSK primitives

- **Polyline:** A polyline is a series of connected line segments. It is defined by a sequence of vertices, where each pair of consecutive vertices is connected by a straight line segment. Polyline primitives

allow you to draw arbitrary shapes and contours by specifying the coordinates of the vertices.

- **Line type:** solid, dashed or dotted.
- **Line width scale factor:** thickness of the line.
- **Polyline color index:** color of the line.

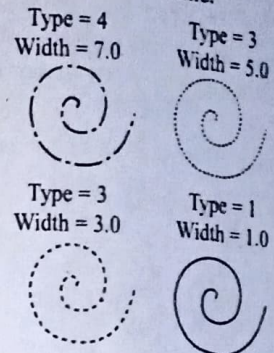


Fig.: GKS polylines

- **Polymarker:** A polymarker is used to draw individual marker symbols at specified locations. It represents a single point in the graphics coordinate system, marked by a marker symbol instead of a line. Polymarkers are often used to highlight specific points of interest or data points in a graph.
- **Marker characters:** dot, plus, asterisk, circle or cross.
- **Marker size scale factor:** size of marker
- Polymarker color of the marker.

Polymarkers

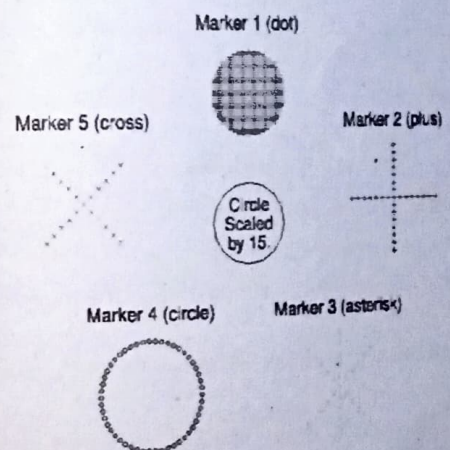


Fig.: GKS polymarkers

- **Fill Area:** The fill area primitive allows you to fill enclosed regions with a specified fill pattern or color. It is commonly used to represent solid filled shapes, such as polygons, with a chosen interior color or pattern. Fill area primitives help create visually distinct regions or emphasize certain parts of a graphical image.
- **Fill area interior style:** solid colors, hatch patterns.
- **Fill area style index:** horizontal lines; vertical lines; left slant lines; right slant lines; horizontal and vertical lines; or left slant and right slant lines.
- **Fill area color index:** color of the fill patterns/solid areas.

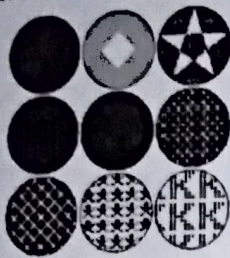


Fig.: GKS Fill area

- **Text:** GKS provides the ability to render text in graphics images. It allows you to position and display alphanumeric characters on the output device. You can specify the font, size, style, and alignment of the text. Text primitives are useful for adding labels, annotations, or textual information to your graphical representation.
- **Text font and precision:** text font should be used for the characters.
- **Character expansion factor:** height-to-width ratio of each character.
- **Character spacing:** additional white space should be inserted between characters.
- **Text color index:** color the text string
- **Character height:** size of the characters
- **Character up vector:** angle the text
- **Text path:** direction the text should be written (right, left, up, or down).
- **Text alignment:** vertical and horizontal centering options for the text string.

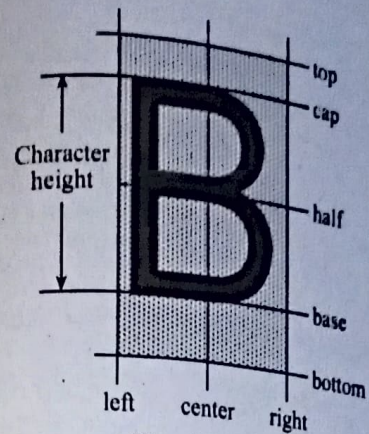


Fig.: GKS Text

- **Cell Array:** A cell array is a rectangular grid of cells, where each cell can contain a predefined pattern or image. GKS supports the representation and rendering of cell arrays as a graphical primitive. Cell arrays are commonly used for textures, shading, or creating repetitive patterns in computer graphics.

Need of machine independent language

- Machine-independent languages in computer graphics ensure cross-platform compatibility.
- They provide hardware abstraction, allowing developers to interact with graphics hardware in a portable manner.
- Machine-independent languages simplify development and increase efficiency by providing high-level abstractions and libraries.
- Graphics code written in a machine-independent language can be reused across different platforms, saving time and effort.
- These languages offer future compatibility by enabling easy adaptation to new hardware technologies and APIs.

Aspect	PHIGS	GKS
Purpose and focus	Interactive graphics with emphasis on	Device-independent programming interface for
	Non-photorealistic rendering and hierarchical	2D and 3D graphics
Graphics model	Hierarchical graphics model	Flat graphics model

Aspect	PHIGS	GKS
Level of detail	Fine-grained control over rendering	Standardized level of control over rendering
Scene Hierarchy	Supports a hierarchical structure of objects	Objects treated as independent entities
Transformation control	Extensive control over transformations	Basic transformation capabilities
Rendering capabilities	Advanced features like shading and hidden surface removal	Standard capabilities rendering
Primary use	CAD (Computer-Aided Design), scientific visualization	General-purpose computer graphics applications
Standardization	ISO 9592	ISO 7942

OpenGL

- OpenGL is a low-level graphics library specification.
- It makes available to the programmer a small set of geometric primitives - points, lines, polygons, images, and bitmaps.
- OpenGL provides a set of commands that allow the specification of geometric objects in two or three dimensions, using the provided primitives, together with commands that control how these objects are rendered (drawn).
- OpenGL is the most extensively documented 3D graphics API (Application Program Interface) to date.
- Information regarding OpenGL is all over the Web and in print. It is impossible to exhaustively list all sources of OpenGL information.
- OpenGL programs are typically written in C and C++.
- One can also program OpenGL from Delphi (a Pascal-like language), Basic, Fortran, Ada, and other languages.
- To compile and link OpenGL programs, one will need OpenGL header files.
- To run OpenGL programs one may need shared or dynamically loaded OpenGL libraries, or a vendor-specific OpenGL Installable Client Driver (ICD).
- Since OpenGL drawing commands are limited to those that generate simple geometric primitives (points, lines, and polygons), the OpenGL Utility Toolkit (GLUT) has been created to aid in the development of more complicated three-dimensional objects such as a sphere, a torus, and even a teapot.

OpenGL features

Graphics API: OpenGL is a widely used open standard graphics API for rendering 2D and 3D computer graphics.

Cross-Platform: It is designed to be platform-independent, enabling developers to create graphics applications that can run on different operating systems and hardware platforms.

Hardware Acceleration: OpenGL leverages the capabilities of graphics hardware, such as GPUs, to accelerate rendering and achieve high-performance graphics.

Rendering Pipeline: It follows a programmable rendering pipeline that consists of various stages, including vertex processing, geometry processing, rasterization, and fragment processing.

Graphics Primitives: OpenGL provides a set of graphics primitives, such as points, lines, and polygons, which allow the creation of complex graphical objects and scenes.

Shaders: It includes a shading language called GLSL (OpenGL Shading Language) that enables developers to write custom shaders for advanced visual effects, lighting, and material properties.

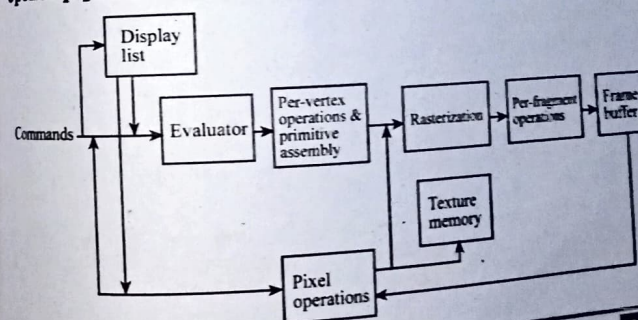
Textures: OpenGL supports the creation and manipulation of textures, which are used to add detailed surface appearance to objects in 3D graphics.

Buffer Objects: It provides buffer objects that allow efficient data storage and manipulation, such as vertex buffers for storing vertex data and texture buffers for storing texture data.

Community Support: OpenGL has a large and active community of developers, offering resources, forums, and libraries that support graphics programming.

Extensibility: OpenGL is extensible, allowing hardware vendors to provide additional functionality through extensions, enabling support for new features and optimizations.

OpenGL pipeline



Display Lists:

- All data, whether it describes geometry or pixels, can be saved in a display list for current or later use.
- (The alternative to retaining data in a display list is processing the data immediately - also known as immediate mode.) When a display list is executed, the retained data is sent from the display list just as if it were sent by the application in immediate mode.

Evaluators:

- All geometric primitives are eventually described by vertices. Parametric curves and surfaces may be initially described by control points and polynomial functions called basis functions. Evaluators provide a method to derive the vertices used to represent the surface from the control points. The method is a polynomial mapping, which can produce surface normal, texture coordinates, colors, and spatial coordinate values from the control points.

Per-Vertex:

- Operations For vertex data, next is the "per-vertex operations" stage, which converts the vertices into primitives.
- Some vertex data (for example, spatial coordinates) are transformed by 4×4 floating-point matrices.
- Spatial coordinates are projected from a position in the 3D world to a position on your screen.
- If advanced features are enabled, this stage is even busier. If texturing is used, texture coordinates may be generated and transformed here.
- If lighting is enabled, the lighting calculations are performed using the transformed vertex, surface normal, light source position, material properties, and other lighting information to produce a color value.

Primitive Assembly:

- Clipping, a major part of the primitive assembly, is the elimination of portions of geometry that fall outside a half-space, defined by a plane.
- Point clipping simply passes or rejects vertices; line or polygon clipping can add additional vertices depending on how the line or polygon is clipped.

- In some cases, this is followed by perspective division, which makes distant geometric objects appear smaller than closer objects. Then viewport and depth (z coordinate) operations are applied. If culling is enabled and the primitive is a polygon, it then may be rejected by a culling test.
- Depending upon the polygon mode, a polygon may be drawn as points or lines.
- The results of this stage are complete geometric primitives, which are the transformed and clipped vertices with related color, depth, and sometimes texture-coordinate values and guidelines for the rasterization step.

Pixel Operations:

- While geometric data takes one path through the OpenGL rendering pipeline, pixel data takes a different route.
- Pixels from an array in system memory are first unpacked from one of a variety of formats into the proper number of components.
- Next, the data is scaled, biased, and processed by a pixel map. The results are clamped and then either written into texture memory or sent to the rasterization step. If pixel data is read from the frame buffer, pixel-transfer operations (scale, bias, mapping, and clamping) are performed.
- Then these results are packed into an appropriate format and returned to an array in system memory.
- There are special pixel copy operations to copy data in the framebuffer to other parts of the framebuffer or to the texture memory. A single pass is made through the pixel transfer operations before the data is written to the texture memory or back to the framebuffer.

Texture Assembly:

- An OpenGL application may wish to apply texture images onto geometric objects to make them look more realistic.
- If several texture images are used, it's wise to put them into texture objects so that you can easily switch among them.
- Some OpenGL implementations may have special resources to accelerate texture performance.
- There may be specialized, high-performance texture memory. If this memory is available, the texture objects may be prioritized to control the use of this limited and valuable resource.

Rasterization:

- Rasterization is the conversion of both geometric and pixel data into fragments. Each fragment square corresponds to a pixel in the framebuffer.

- Line and polygon stipples, line width, point size, shading model, and coverage calculations to support antialiasing are taken into consideration as vertices are connected into lines or the interior pixels are calculated for a filled polygon.
- Color and depth values are assigned for each fragment square.

Fragment Operations:

- Before values are actually stored into the framebuffer, a series of operations are performed that may alter or even throw out fragments.
- All these operations can be enabled or disabled. The first operation which may be encountered is texturing, where a texel (texture element) is generated from texture memory for each fragment and applied to the fragment.
- Then fog calculations may be applied, followed by the scissor test, the alpha test, the stencil test, and the depth-buffer test (the depth buffer is for hidden-surface removal). Failing an enabled test may end the continued processing of a fragment's square. Then, blending, dithering, logical operation, and masking by a bitmask may be performed.
- Finally, the thoroughly processed fragment is drawn into the appropriate buffer, where it has finally advanced to be a pixel and achieved its final resting place.

GLUT

- The OpenGL Utility Toolkit (GLUT) is a library of utilities for OpenGL programs, which primarily perform system-level I/O with the host operating system.
- Functions performed include window definition, window control, and monitoring of keyboard and mouse input.
- Routines for drawing a number of geometric primitives (both in solid and wireframe mode) are also provided, including cubes, spheres, and cylinders.
- GLUT even has some limited support for creating pop-up menus.
- The two aims of GLUT are to allow the creation of rather portable code between operating systems (GLUT is cross-platform) and to make learning OpenGL easier. All GLUT functions start with the glut prefix (for example, glutPostRedisplay marks the current window as needing to be redrawn).

Setting up GLUT - main() GLUT provides high-level utilities to simplify OpenGL programming, especially in interacting with the Operating System (such as creating a window, handling key and

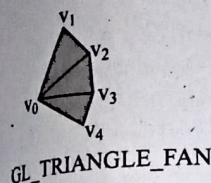
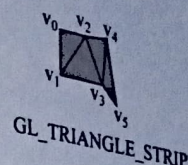
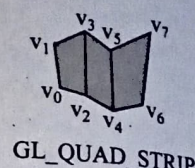
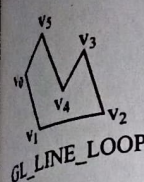
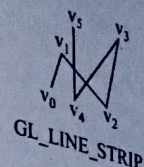
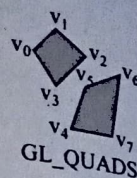
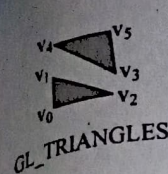
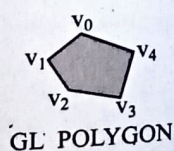
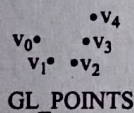
mouse inputs). The following GLUT functions were used in the above program:

- **glutInit:** initializes GLUT, must be called before other GL/GLUT functions. It takes the same arguments as the main(). void glutInit(int *argc, char **argv)
- **glutCreateWindow:** creates a window with the given title. int glutCreateWindow(char *title)
- **glutInitWindowSize:** specifies the initial window width and height, in pixels. void glutInitWindowSize(int width, int height)
- **glutInitWindowPosition:** positions the top-left corner of the initial window at (x, y). The coordinates (x, y), in term of pixels, is measured in window coordinates, i.e., origin (0, 0) is at the top-left corner of the screen; x-axis pointing right and y-axis pointing down. void glutInitWindowPosition(int x, int y)
- **glutDisplayFunc:** registers the callback function (or event handler) for handling window-paint event. The OpenGL graphic system calls back this handler when it receives a window repaint request. In the example, we register the function display() as the handler. void glutDisplayFunc(void (*func)(void))
- **glutMainLoop:** enters the infinite event-processing loop, i.e., put the OpenGL graphics system to wait for events (such as repaint), and trigger respective event handlers (such as display()). void glutMainLoop()
- **glutInitDisplayMode:** requests a display with the specified mode, such as color mode (GLUT_RGB, GLUT_RGBA, GLUT_INDEX), single/double buffering (GLUT_SINGLE, GLUT_DOUBLE), enable depth (GLUT_DEPTH), joined with a bit OR '|'. void glutInitDisplayMode(unsigned int displayMode)
- **void glMatrixMode (GLenum mode);** The glMatrixMode function specifies which matrix is the current matrix.
- **Void**
glOrtho(GLdoubleleft, GLdouble right, GLdouble bottom, GLdouble top, GLdouble zNear, GLdouble zFar); The glOrtho function multiplies the current matrix by an orthographic matrix.
- **void glPointSize (GLfloat size);** The glPointSize function specifies the diameter of rasterized points.
- **void glutPostRedisplay(void);** glutPostRedisplay marks the current window as needing to be redisplayed.
- **void glPushMatrix (void); void glPopMatrix (void);** The glPushMatrix and glPopMatrix functions push and pop the current matrix stack

- **GLint glRenderMode (GLenum mode);** The **glRenderMode** function sets the rasterization mode.
- **void glRotatef (GLfloat angle, GLfloat x, GLfloat y, GLfloat z);** The **glRotatef** functions multiply the current matrix by a rotation matrix.
- **void glScalef (GLfloat x, GLfloat y, GLfloat z);** The **glScalef** functions multiply the current matrix by a general scaling matrix.
- **void glTranslatef (GLfloat x, GLfloat y, GLfloat z);** The **glTranslatef** functions multiply the current matrix by a translation matrix.
- **void glViewport (GLint x, GLint y, GLsizei width, GLsizei height);** The **glViewport** function sets the viewport.
- **void glEnable, glDisable();** The **glEnable** and **glDisable** functions enable or disable OpenGL capabilities.
- **glutBitmapCharacter();** The **glutBitmapCharacter** function used for font style

Open GL Primitives

Value	Meaning
GL_POINTS	individual points
GL_LINES	pairs of vertices interpreted as individual line segments
GL_POLYGON	boundary of a simple, convex polygon
GL_TRIANGLES	triples of vertices interpreted as triangles
GL_QUADS	quadruples of vertices interpreted as four-sided polygons
GL_LINE_STRIP	series of connected line segments
GL_LINE_LOOP	same as above, with a segment added between last and first vertices
GL_TRIANGLE_STRIP	linked strip of triangles
GL_TRIANGLE_FAN	linked fan of triangles
GL_QUAD_STRIP	linked strip of quadrilaterals



OpenGL Implementations:

OpenGL IS an API (think of as collection of .h files):

```
#include <GL/gl.h>
#include <GL/glu.h>
#include <GL/glut.h>
```

Windows, Linux, UNIX, etc. all provide a platform specific implementation. Windows: opengl32.lib glu32.lib glut32.lib Linux: -l GL -l GLU -l GLUT.

Setup

- We need a C/C++ compiler, either GCC (GNU Compiler Collection) from MinGW or Cygwin (for Windows), or Visual C/C++ Compiler, or others.
- We need the following sets of libraries in programming OpenGL:
 - **Core OpenGL (GL):** consists of hundreds of functions, which begin with a prefix "gl" (e.g., glColor, glVertex, glTranslate, glRotate). The Core OpenGL models an object via a set of geometric primitives, such as point, line, and polygon.
 - **OpenGL Utility Library (GLU):** built on-top of the core OpenGL to provide important utilities and more building models (such as

quadric surfaces). GLU functions start with a prefix "glu" (e.g., gluLookAt, gluPerspective).

- **OpenGL Utilities Toolkit (GLUT):** provides support to interact with the Operating System (such as creating a window, handling key and mouse inputs); and more building models (such as sphere and torus). GLUT functions start with a prefix of "glut" (e.g., glutCreateWindow, glutMouseFunc).

- **Quoting from the opengl.org:** "GLUT is designed for constructing small to medium sized OpenGL programs. While GLUT is well-suited to learning OpenGL and developing simple OpenGL applications, GLUT is not a full-featured toolkit so large applications requiring sophisticated user interfaces are better off using native window system toolkits. GLUT is simple, easy, and small."

- Alternative of GLUT includes SDL.

- **OpenGL Extension Wrangler Library (GLEW):** "GLEW is a cross-platform open-source C/C++ extension loading library. GLEW provides efficient run-time mechanisms for determining which OpenGL extensions are supported on the target platform." Source and pre-build binary available at <http://glew.sourceforge.net/>.

- Each of the software package consists of:

- **A header file:** "gl.h" for core OpenGL, "glu.h" for GLU, and "glut.h" (or "freeglut.h") for GLUT, typically kept under "include/GL" directory.
- **A static library:** for example, in Win32, "libopengl32.a" for core OpenGL, "libglu32.a" for GLU, "libglut32.a" (or "libfreeglut.a" or "glut32.lib") for GLUT, typically kept under "lib" directory.
- **An optional shared library:** for example, "glut32.dll" (for "freeglut.dll") for GLUT under Win32, typically kept under "bin" or "c:\windows\system32".

Sample program

```
#include<windows.h>
#include<GL/glut.h>
void Myinit()
{
    glClearColor(0,0,0,0.5);
    glColor3f(1,0,0);
}
```

```
void display()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glPointSize(5);
    glBegin(GL_POLYGON);
    glVertex2f(-0.5,0.5);
    glVertex2f(0.5,0.5);
    glVertex2f(0.5,-0.5);
    glVertex2f(-0.5,-0.5);
    glEnd();
    glFlush();
}
```

```
int main(int argc,char** argv)
{
    glutInit(&argc,argv); //Initialize GLUT
    glutInitWindowSize(500,500); //to increase or decrease the size of
    the windows
    glutInitWindowPosition(150,50);
    glutCreateWindow("Lab 1"); //create a window with the title lab1;
    Myinit();
    glutDisplayFunc(display); //to display
    glutMainLoop();
}
```

